MÉTHODE DE TRAVAIL · DISCUSSION AVEC CLAUDE.AI

Claude.ai : 5 étapes de discussion pour ce projet

Outil utilisé : **Claude.ai**. On discute par prompts, on copie le code dans le projet, puis on exécute les tests localement. Cinq étapes suffisent pour construire et vérifier la démo.

Règle : Claude.ai ne remplace pas la vérification. Il propose, explique et corrige ; l'étudiant colle le code, lance compileall + pytest et assume les résultats.

Étape	Prompt à envoyer à Claude.ai	Résultat attendu / vérification
1. Cadrer	Tu es mon binôme pour un moteur de recherche sémantique sur mes notes. Stack : Python, sentence-transformers, FAISS, Streamlit, pytest. Ne code pas encore : propose l'arborescence, les contrats, les tests à écrire et les limites.	Plan + fichiers vides + tests skipped. Vérifier que la structure correspond au sujet.
2. Coder le moteur	Implémente src/module.py : charge les notes, découpe en passages de 300 mots avec 50 de chevauchement, calcule les embeddings MiniLM, utilise FAISS sinon NumPy, retourne des SearchResult triés. Gère corpus vide, requête vide et top_k invalide.	Code + tests. Lancer pytest et vérifier les scores sur une requête réelle.
3. Évaluer	Écris les tests et src/evaluate.py : Recall@k, Precision@k, MRR et baseline mots-clés. Utilise un petit jeu de requêtes annotées. Les tests doivent échouer si le contrat casse.	Métriques mesurées, pas inventées. Comparer sémantique et mots-clés.
4. Interface	Crée demo_app.py : notes d'exemple chargées automatiquement, upload JSON/TXT/MD, slider du nombre de résultats, compteurs Notes/Passages, recherche et évaluation. Si l'écran disparaît, corrige sans supprimer la recherche.	Application Streamlit. Tester sans fichier puis avec un fichier importé.
5. Vérifier et préparer l'oral	Agis comme un relecteur critique : liste les affirmations non prouvées, les tests vides et les limites. Explique le pipeline en 5 minutes, puis pose-moi 5 questions pièges.	Rapport final + réponses orales. Corriger les écarts avant de rendre.

Ce qu'il faut retenir

- Les prompts sont courts mais précis : objectif, contraintes, livrable, vérification.
- Le moteur de recherche utilise sentence-transformers ; Claude.ai sert à produire et expliquer le code.
- Un chiffre ou un test ne se copie jamais sans avoir été exécuté sur la machine.
- Garder une trace des prompts et des réponses dans le dossier de projet.